

Unit: III
Physical Devices
Lecture: 2
I/O Buffering

A buffer is an intermediate memory area under operating system control that stores data in transit between two devices or between user's work area and device. It allows computation to proceed in parallel with I/O.

In a typical unbuffered transfer situation the processor is idle for most of the time, waiting for data transfer to complete and total read-processing time is sum of all the transfer/ read time and processor time.

Purpose of Buffering:

You face buffer during watching videos on YouTube or live streams. In a video stream, a buffer represents the amount of data required to be downloaded before the video can play to the viewer in real-time. A buffer in a computer environment means that a set amount of data will be stored to preload the required data before it gets used by the CPU.

Computers have many different devices that operate at varying speeds, and a buffer is needed to act as a temporary placeholder for everything interacting.

This is done to keep everything running efficiently and without issues between all the devices, programs, and processes running at that time.

There are three reasons behind buffering of data:

1. It helps in **matching speed** between two devices in which the data is transmitted. For example, a hard disk has to store the file received from the modem. As we know, the transmission speed of a modem is slow compared to the hard disk. So bytes coming from the modem is accumulated in the buffer space, and when all the bytes of a file has arrived at the buffer, the entire data is written to the hard disk in a single operation.
2. It helps the devices with different **sizes of data transfer** to get adapted to each other. It helps devices to manipulate data before sending or receiving it. In computer networking, the large message is fragmented into small fragments and sent over the network. The fragments are accumulated in the buffer at the receiving end and reassembled to form a complete large message.

3. It also supports **copy semantics**. With copy semantics, the version of data in the buffer is guaranteed to be the version of data at the time of system call, irrespective of any subsequent change to data in the buffer. Buffering increases the performance of the device. It overlaps the I/O of one job with the computation of the same job.

- **Single-buffered transfer:** In case of single-buffered transfer, **blocks are first read into a buffer and then moved to the user's work area**. When the move is complete, the next block is read into the buffer and processed in parallel with the first block. This helps in minimizing speed mismatch between devices and the processor. Also, this allows process computation in parallel with input/output.

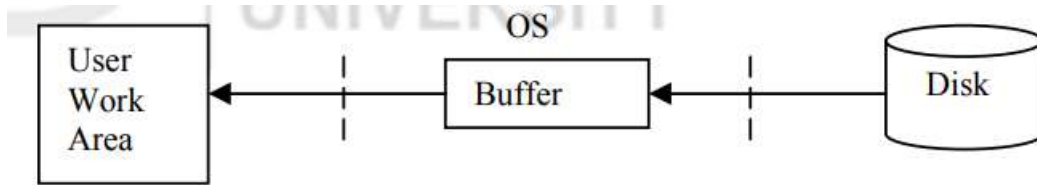


Fig: Single Buffering

- **Double Buffering:** Double buffering is an improvement over this. A pair of buffers is used. Blocks/records generated by a running process are initially stored in the first buffer until it is full. Then from this buffer it is transferred to the secondary storage. During this transfer the other blocks generated are deposited in the second buffer and when this second buffer is also full and first buffer transfer is complete, then transfer from the second buffer is initiated. This process **of alternation between buffers continues which allows I/O to occur in parallel with a process's computation**. This scheme increases the complexity but yields improved performance.